

INSTITUTO FEDERAL
Sul-rio-grandense

Câmpus
Pelotas



REVISÃO DE ORIENTAÇÃO A OBJETO Parte 2



Paradigma de POO Aplicada a Linguagem PHP



CSTSI - IV - SEMESTRE
DESENVOLVIMENTO BACK-END 2
Prof. Gill Velleda Gonzales | gillgonzales@ifsul.edu.br
Agosto 2025

O que vamos (re)ver



Configuração do ambiente e ferramentas

- Utilização do composer
- **Tópicos sobre POO aplicados a linguagem PHP:**
 - Namespace, Classes e Objetos;
 - Métodos, Construtor e Destrutor;
 - Modificadores de acesso, Getters e Setters;
 - Herança e Traits
 - Classe Abstrata e Interfaces
 - Polimorfismo
 - Exceptions e Try/Catch
- **Atividades práticas**



Composer

Gerenciador de dependências do PHP



- Gerenciador de **DEPENDÊNCIAS** do PHP:
- E o que são **Dependências** ?

São programas de terceiros que compoem o sistema e que ele depende para funcionar corretamente. Desta forma não programamos tudo do zero.

- E como sei **quais** são as **Dependências**?

Através de um gerenciador de pacotes/dependências, como o já conhecido **composer** para o **php**. Ou ainda **npm**, **yarn**, **pnpm** para **nodejs**, **pip** para **python**, **apt** para distribuições *Linux* baseadas em *Debian*, **rpm** para as demais baseadas em CentOS, ou **rubygems** para **ruby**.

- Repositório de pacotes PHP: <https://packagist.org/>





Primeiro Instalamos/Atualizamos o Composer

Visite o site do composer: <https://getcomposer.org/download/>

Siga as instruções para instalação no seu ambiente de desenvolvimento

- Exemplo para Linux:

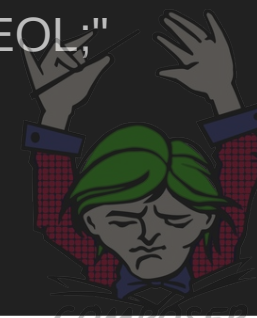
Comandos:

```
$> php -r "copy('https://getcomposer.org/installer', 'composer-setup.php');"
```

```
$> php -r "if (hash_file('sha384', 'composer-setup.php') ===  
'906a84df04cea2aa72f40b5f787e49f22d4c2f19492ac310e8cba5b96ac8b64115ac402c8cd292b8a03482574915d1  
a8') { echo 'Installer verified'; } else { echo 'Installer corrupt'; unlink('composer-setup.php'); } echo PHP_EOL;"
```

```
$> php composer-setup.php
```

```
$> php -r "unlink('composer-setup.php');"
```





Dos quatro comandos anteriores:

- 1) O primeiro comando usa o próprio interpretador do php para baixar a nova versão do composer.
- 2) O instalador será baixado do site para o diretório em que você está trabalhando.
- 3) Logo, o segundo comando verifica a validade do arquivo de instalação de acordo com a *hash* fornecida.
- 4) O terceiro comando executa o instalador do composer (***composer-setup.php***) com o interpretador do php.
- 5) O quarto e último comando exclui o instalador!





Composer instalado !

```
All settings correct for using Composer
Downloading...
```

```
Composer (version 2.1.14) successfully installed to: /me
dia/Dados/Projetos/composer.phar
Use it: php composer.phar
```



Composer

Gerenciador de dependências do PHP



Diretório de instalação:

```
Composer (version 2.1.14) successfully installed to: /me  
dia/Dados/Projetos/composer.phar  
Use it: php composer.phar
```

No entanto, note que o composer foi instalado no meu diretório de projetos.

Para isso não ocorrer você pode executar o instalador com o seguinte parâmetro:

```
php composer-setup.php --install-dir=/usr/local/bin/composer
```

Ou, após instalado, corrija o diretório de instalação executando o seguinte comando:

```
sudo mv composer.phar /usr/local/bin/composer
```

Composer

Gerenciador de dependências do PHP



- Links de instalação:
- Windows:
 - <https://getcomposer.org/doc/00-intro.md#installation-windows>
 - Tutorial: [link](#)
- Linux e Macs:
 - PHP: <https://getcomposer.org/download/>
 - Instruções:
<https://getcomposer.org/doc/00-intro.md#installation-linux-unix-macos>



Composer

Gerenciador de dependências do PHP



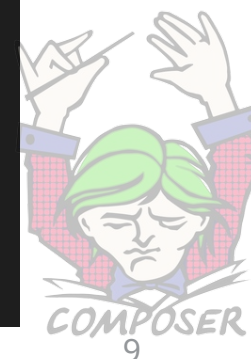
\$>composer

```
...tsi_daoo/cstsi_daoo_20222_proto_poo/aula_03_pdo_mvc > <?php 8.1.12
poser

Composer version 2.1.14 2021-11-30 10:51:43

Usage:
  command [options] [arguments]

Options:
  -h, --help                Display this help message
  -q, --quiet               Do not output any message
  -V, --version             Display this application version
      --ansi                Force ANSI output
      --no-ansi             Disable ANSI output
```





CRIAÇÃO DE UM NOVO PROJETO NO DIRETÓRIO ATUAL

```
$>mkdir project_folder
```

```
$>cd project_folder
```

```
$>composer init
```



```
o > composer ini
```

```
Welcome to the Composer config generator
```

```
This command will guide you through creating your composer.json config.
```

```
Package name (<vendor>/<name>) [gillgonzales/cstsi_202501_dbe2_php_laravel]:
```



CRIAÇÃO DE UM NOVO PROJETO NO DIRETÓRIO ATUAL

- Após responder as perguntas que configuram o projeto, o composer criará um arquivo chamado ***composer.json***, com as configurações desejadas.

```
{ } composer.json > ...
1  {
2      "name": "gill/aula_02",
3      "description": "Projeto com exemplos da aula 02 DA00-20241",
4      "type": "project",
5      "require-dev": {
6          "psy/psysh": "@stable"
7      },
8      "license": "MIT",
9      "autoload": {
10         "psr-4": {
11             "Daoo\\Aula01Projeto02\\": "src/"
12         }
13     }
```





CRIAÇÃO DO PROJETO COM APENAS UM COMANDO:

- Também é possível passar um comando apenas com as principais configurações de nome (**--name**), autoload (**-a**) e/ou dependências (**--require-dev**).
- Para não solicitar interação (**-n**) e não gerar saída também utilizamos os comandos **--quiet -n**:

```
~/Projetos/ifsul/cstsi/daoo/202402/daoo/prototipos/aula02/projeto1
> composer init --quiet --name daoo202402/exemplos_poo_01 -a src --require-dev psy/psysh:@stable

~/Projetos/ifsul/cstsi/daoo/202402/daoo/prototipos/aula02/projeto1
> ls
composer.json  src
```

\$>composer init --quiet --name daoo202402/exemplos_poo_01 -a src --require-dev psy/psysh:@stable

- **Vendor: daoo202404**
- **Projeto: exemplos_poo_01**





INSTALAÇÃO DE DEPENDÊNCIAS EM UM PROJETO EXISTENTE

\$>composer install

\$>composer i

```
> ls
composer.json  composer.lock  src

~/Projetos/ifsul/cstsi/daoo/202401/cstsi_20241_daoo_php_laravel/aula_02 main
> composer i
Installing dependencies from lock file (including require-dev)
Verifying lock file contents can be installed on current platform.
Package operations: 11 installs, 0 updates, 0 removals
- Downloading symfony/var-dumper (7.1.x-dev c431188)
- Downloading symfony/string (7.1.x-dev ef96485)
- Downloading psr/container (dev-master 7079847)
- Downloading symfony/service-contracts (dev-main cea2ecc)
- Downloading symfony/console (7.1.x-dev 31c74a5)
- Downloading psy/psysh (v0.12.0)
```





INSTALAÇÃO DE DEPENDÊNCIAS EM UM PROJETO EXISTENTE

- Também é possível adicionar novas dependências (***require***), após a criação do projeto.

```
• > composer require vlucas/phpdotenv
./composer.json has been updated
Running composer update vlucas/phpdotenv
Loading composer repositories with package information
Updating dependencies
Lock file operations: 4 installs, 0 updates, 0 removals
  - Locking graham-campbell/result-type (v1.1.3)
  - Locking phoption/phoption (1.9.4)
  - Locking symfony/polyfill-php80 (v1.33.0)
  - Locking vlucas/phpdotenv (v5.6.2)
```

- `$>composer require vendor/packageName`





CONFIGURAÇÃO DE SCRIPTS DO PROJETO:

- Também é possível adicionar novas dependências (***require***), após a criação do projeto.

```
"scripts":{  
    "start":"php -S localhost:8081 -t src",  
    "psy": "./vendor/psy/psysh/bin/psysh"  
},  
"require": {  
    "psy/psysh": "@stable",  
    "vlucas/phpdotenv": "^5.6",  
    "illuminate/support": "^12.26"  
},  
"config": {  
    "process-timeout":0  
}  
}
```

- \$>composer require vendor/packageName**



Carregamento automático

Revisão de padrões PHP e Gerenciamento de pacotes com Composer.



- **Autoloading**: o php possui a função **spl autoload register** ao qual passamos um **callback** que tratará das inclusões das classes do sistema.
- Eliminamos a necessidade de usar vários include/require em nossa aplicação.

```
src >  exemplo_01_spl_autoload.php > ...  
1  <?php  
2  spl_autoload_register(function ($class_name){  
3      echo "\nautoload_namespace:\n";  
4      var_dump($class_name);  
5      echo "\n\tDIR: ".__DIR__."\n\n";  
6  
7      $path = implode('/',explode('\\',$class_name));  
8      echo "\n\t$path\n\n";  
9      require_once "$path.php";  
10 } );|
```

- Com o **composer**, utilizaremos a configuração padrão **autoload** que implementa a recomendação **PSR-4** para carregamento de arquivos.

Carregamento automático

Revisão de padrões PHP e Gerenciamento de pacotes com Composer.



- Com o **PSR-4** o composer irá mapear um namespace do projeto para uma pasta específica no **composer.json**, geralmente é a própria **src**.

```
5      "require-dev": {
6          "psy/psysh": "@stable"
7      },
8      "license": "MIT",
9      "autoload": {
10         "psr-4": {
11             "Daoo\\Aula01Projeto02\\": "src/"
12         }
13     },
14     "scripts": {
```

- Então toda vez que se usar o **namespace** **Daoo\\Aula01Projeto02**, a classe será procurada na pasta **src**.

Carregamento automático

Revisão de padrões PHP e Gerenciamento de pacotes com Composer.



- Aqui a classe **Atleta** será encontrada de acordo com o **path (\)** definido no **use**.

```
src
├── classes
├── interfaces
├── traits
└── exemplo_01_auto...
```

```
3
4  use Daoo\Aula01Projeto02\classes\Atleta;
5
6  $atl1 = new Atleta("Luizito",36,1.8,80);
7
8  $atl1->showImc();
```

- E a própria classe também deverá ser delcarada com o mesmo **namespace**.

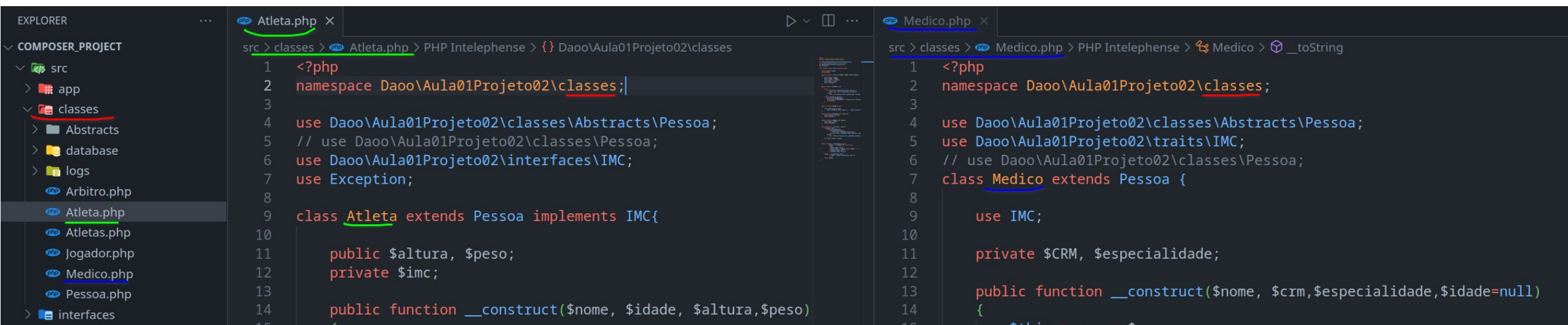
```
1  <?php
2  namespace Daoo\Aula01Projeto02\classes;
3
4  use Daoo\Aula01Projeto02\classes\Pessoa;
5  use Exception;
6
7  class Atleta extends Pessoa{
```

Namespaces e use

Revisão de padrões PHP e Gerenciamento de pacotes com Composer..



- Em PHP podemos organizar nossas classes em diferentes grupos chamados de **namespaces**, como se fossem diretórios ou subdiretórios.
- Desta forma, evita-se conflito de nomes de classes semelhantes.



```
1 <?php
2 namespace Daoo\Aula01Projeto02\classes;
3
4 use Daoo\Aula01Projeto02\classes\Abstracts\Pessoa;
5 // use Daoo\Aula01Projeto02\classes\Pessoa;
6 use Daoo\Aula01Projeto02\interfaces\IMC;
7 use Exception;
8
9 class Atleta extends Pessoa implements IMC{
10
11     public $altura, $peso;
12     private $imc;
13
14     public function __construct($nome, $idade, $altura,$peso)
15
```

```
1 <?php
2 namespace Daoo\Aula01Projeto02\classes;
3
4 use Daoo\Aula01Projeto02\classes\Abstracts\Pessoa;
5 use Daoo\Aula01Projeto02\traits\IMC;
6 // use Daoo\Aula01Projeto02\classes\Pessoa;
7 class Medico extends Pessoa {
8
9     use IMC;
10
11     private $CRM, $especialidade;
12
13     public function __construct($nome, $crm,$especialidade,$idade=null)
14     {
15         $this->nome = $nome;
16
```

- Apesar de serem muito utilizados para classes, **namespaces** podem ser usados tanto em funções como em CONSTANTES.
- A ideia é resolver conflitos de nomes em sistemas com muitos códigos reutilizáveis baseados em pacotes ou bibliotecas.

Namespaces e *use*

Revisão de conceitos básicos de POO aplicados ao PHP.



- Para “**importar**” as classes que precisamos utilizar em **namespaces** diferentes usamos a palavra reservada ***use***.

```
src > classes >  Atleta.php > ...  
1  <?php  
2  namespace Daoo\Aula01Projeto02\classes;  
3  
4  use Daoo\Aula01Projeto02\classes\Pessoa;  
5  use Daoo\Aula01Projeto02\interfaces\IMC;  
6  use Exception;  
7  
8  class Atleta extends Pessoa implements IMC{  
9
```

- Observe que as classes **Pessoa** e **IMC** estão em pastas diferentes. **Daoo\Aula01Projeto02** está apontado para a pasta **src**.

Namespaces e *use*

Revisão de conceitos básicos de POO aplicados ao PHP.



- Exemplo de uso de apelidos com o ***use*** e ***as***.

```
use Dbe2\Aula02\classes\Atleta;  
use Dbe2\Aula02\classes\Medico;  
use Illuminate\Support\Str;  
  
$atl1 = new Atleta("Pedro Geromel",36,1.83,75);  
$med1 = new Medico("Pualo Paixão",122343,"Fisioterapeuta");  
  
$list = [$atl1,$med1];  
  
var_dump($list);  
  
print_r(array_map(fn($p)=>Str::slug($p->nome),$list));
```

- Obs:*** a lib *illuminate/support* tem vários métodos ***estáticos***, ou seja, chamados a partir da classe com operador de escopo ***::*** sem a necessidade de uma instância;

Namespaces e *use*

Revisão de conceitos básicos de POO aplicados ao PHP.



- Combinando o **use** com o **as** criamos um **alias**(apelidos) para as classes se diferenciarem de outras se necessário.

```
src > php exemplo_04_abstract.php > ...  
1  <?php  
2  
3  echo "<pre>";  
4  
5  use Daoo\Aula01Projeto02\classes\Atleta;  
6  use Daoo\Aula01Projeto02\classes\Medico;  
7  use Daoo\Aula01Projeto02\classes\Abstracts\Pessoa as PessoaAbstrata;  
8  use Daoo\Aula01Projeto02\classes\Pessoa;  
9  use Daoo\Aula01Projeto02\classes\logs\Relatorio;  
10  
11  $atl1 = new Atleta("Luizito",36,1.8,80);  
12  $med1 = new Medico("Pualo Paixão",122343,"Fisioterapeuta");  
13  
14  echo "\nAtleta " . (($atl1 instanceof PessoaAbstrata)?"eh":"não eh")." PessoaAbstrata";  
15  echo "\nMédico " . ((!$med1 instanceof Pessoa)?"não":"")." eh Pessoa";
```


Namespaces e *use*

Revisão de conceitos básicos de POO aplicados ao PHP.



- Exemplo do uso de **namespace** em um *framework* como o **Laravel**:

```
1  <?php
2
3  namespace App\Exceptions;
4
5  use Illuminate\Foundation\Exceptions\Handler as ExceptionHandler;
6  use Symfony\Component\HttpKernel\Exception\NotFoundHttpException;
7  use Throwable;
8
9  class Handler extends ExceptionHandler
10 {
```

- No momento este exemplo é apenas para ilustrar um uso prático de **namespaces**, **use** e **as**;

Sobrescrita de Métodos

Revisão de conceitos básicos de POO aplicados ao PHP – Parte 2



- A sobrescrita de métodos (**overriding**) é a característica de uma classe filha reimplementar um método da classe **Pai**, mantendo a **mesma assinatura (passagem de parâmetros)**.
- Exemplo: método mágico **__toString** na classe **Pessoa**

```
1 public function __toString()
2 {
3     return "\n===Dados da Pessoa==="
4         . "\nNome: $this->nome"
5         . ($this->idade ? "\nIdade: $this->idade" : "")
6         . "\nPeso: $this->peso"
7         . "\nAltura: $this->altura";
8 }
```

Sobrescrita de Métodos

Revisão de conceitos básicos de POO aplicados ao PHP – Parte 2



- O método mágico **__toString** é chamado quando forçamos a conversão do objeto para uma string, como no comando **echo**.
- Exemplo: método mágico **__toString** na classe **Atleta**

```
src > classes >  Atleta.php > PHP Intelephense >  Atleta >  __construct
```

```
9  class Atleta extends Pessoa implements IMC{
68      public function __toString():string {
69          $saida = "\n===Dados do ".self::class
70              . "==="
71              . "\nNome: $this->nome"
72              . ($this->idade ? "\nIdade: $this->idade" : "")
73              . "\nPessoa: $this->peso"
74              . "\nAltura: $this->altura";
75      }
```

- Podemos utilizar o **parent::** para reaproveitar o retorno do método **__toString** da classe **Pessoa**.

Classe Abstrata

Revisão de conceitos básicos de POO aplicados ao PHP.



- Classes Abstratas **não podem ser instanciadas**, servem para serem apenas herdadas por outras classes específicas.
- Se uma classe possui um **método abstrato**, deverá ser **abstrata** também.
- Métodos **privados** não podem ser abstratos.
- Uma classe Abstrata pode conter métodos não abstratos ou **apenas** métodos não abstratos (**implementados**).
- As subclasses devem **implementar todos os métodos** abstratos, **caso contrário** deverá ser declarada **abstrata**.

Classe Abstrata

Revisão de conceitos básicos de POO aplicados ao PHP.



- Exemplo da classe Pessoa em transformada em classe Abstrata:

```
1  <?php
2  namespace classes;
3
4  abstract class Pessoa{
5      public $nome, $idade;
6
7      public abstract function __toString();
8
9      public function __destruct()
10     {
11         echo "\n$this->nome foi destruído!\n";
12     }
13
```


Classe Abstrata

Revisão de conceitos básicos de POO aplicados ao PHP.



- Exemplo de classes que herdam da classe Pessoa: **Atleta**

```
1  <?php
2
3  namespace classes;
4
5  class Atleta extends Pessoa
6  {
7      private $peso, $altura;
8
9      public function __construct($nome, $altura, $peso, $idade=null)
10     {
11         $this->nome = $nome;
12         $this->idade = $idade;
13         $this->peso = $peso;
14         $this->altura = $altura;
15         $this->calcImc();
16     }
```

Classe Abstrata

Revisão de conceitos básicos de POO aplicados ao PHP.



- A classe **Atleta** implementa o método abstrato **__toString** da classe **Pessoa**:

```
1 public function __toString()  
2 {  
3     $className = self::class;  
4     return "\n===Dados de $className ==="  
5         . "\nNome: $this->nome"  
6         . ($this->idade ? "\nIdade: $this->idade" : "")  
7         . "\nPeso: $this->peso"  
8         . "\nAltura: $this->altura"  
9         . "\nIMC: " . number_format($this->imc, 2) . "\n";  
10 }
```

-  Exemplo completo da classe **Atleta**.

Classe Abstrata

Revisão de conceitos básicos de POO aplicados ao PHP.



- A classe Médico também é filha da classe Pessoa
- O método **__toString** será implementado de forma diferente:

```
1 public function __toString()  
2 {  
3     $className = self::class;  
4     return "\n===Dados de $className ==="  
5         . "\nNome: $this->nome"  
6         . ($this->idade ? "\nIdade: $this->idade" : "")  
7         . "\nEspecialidade: $this->especialidade"  
8         . "\nCRM: $this->crm\n";  
9 }
```

-  **Exemplo completo da classe Médico.**

Classe Abstrata

Revisão de conceitos básicos de POO aplicados ao PHP.



- **Exemplo de uso:**
- A classe **Relatório** possui uma lista de pessoas que deve imprimir em um relatório.
- Em seu método **log** irá chamar o método **__toString** do objeto do tipo **Pessoa**.
- E no método **imprime** deverá gerar um relatório com os dados de todas as **Pessoas** registradas.
- A classe **Relatório** não precisa saber a especialização do objeto, apenas que ele é do tipo **Pessoa** e deve implementar o método **__toString**.
-  **Exemplo completo da classe Relatório.**

Classe Abstrata

Revisão de conceitos básicos de POO aplicados ao PHP.



- Exemplo dos métodos *log* e *imprime* da classe *Relatório*:

```
1 public function log(Pessoa $pessoa)
2 {
3     echo "\nlog: ".$pessoa;
4 }
5
6 public function imprime(){
7     echo "\n### RELATORIO ###\n";
8     foreach ($this->pessoas as $pessoa) $this->log($pessoa);
9     echo "\n#####\n";
10 }
```

-  Exemplo completo da classe *Relatório*.

Classe Abstrata

Revisão de conceitos básicos de POO aplicados ao PHP.



- Exemplo das classes

```
src >  exemplo_04_abstract.php > ...
 8  use Daoo\Aula01Projeto02\classes\Pessoa;
 9  use Daoo\Aula01Projeto02\classes\logs\Relatorio;
10
11  $atl1 = new Atleta("Luizito",36,1.8,80);
12  $med1 = new Medico("Pualo Paixão",122343,"Fisioterapeuta");
13
14  echo "\nAtleta " . (($atl1 instanceof PessoaAbstrata)?"eh":"não eh")." PessoaAbstrata";
15  echo "\nMédico " . ((!$med1 instanceof Pessoa)?"não":"")." eh Pessoa";
16
17  $relatorio = new Relatorio;
18
19  $relatorio->add($atl1); //Relatório recebe qualquer objeto que herde PessoaAbstrata
20  $relatorio->add($med1);
21  $relatorio->imprime();
22
```

Classe Abstrata

Revisão de conceitos básicos de POO aplicados ao PHP.

- Exemplo da saída do método *imprime* da classe **Relatório**.
- Note que a classe **Relatório** não precisa saber quem são os objetos, apenas que sejam do tipo **Pessoa**.
- Mesma não tendo contato com a especialidade de cada objeto a classe consegue mostrar os dados específicos de **Médico** ou **Atleta**.

```
Atleta eh PessoaAbstrata
Médico não eh Pessoa
### RELATORIO ###
```

log:

```
===Dados do Daoo\Aula01Projeto02\classes\Atleta===
Nome: Luizito
Idade: 36
Pessoa: 80
Altura: 1.8
IMC: 24.691
```

log:

```
=== Dados de Daoo\Aula01Projeto02\classes\Medico ===
Herança: Daoo\Aula01Projeto02\classes\Abstracts\Pessoa
Atributos:
    Nome: Pualo Paixão
    Especialidade: Fisioterapeuta
    CRM: 122343
```

```
#####
```

```
Pualo Paixão foi destruido!
Luizito foi destruido!
```

Classe Abstrata

Revisão de conceitos básicos de POO aplicados ao PHP.

- Rodamos este exemplo no servidor embutido do **PHP**.

```
$>composer run start
```

- **start** esta configurado como uma chave para chamar os comandos “**php -S ...**”.
- **psy**: inicia o REPL pra psyshel para PHP.

```
Atleta eh PessoaAbstrata
Médico não eh Pessoa
### RELATORIO ###
```

log:

```
===Dados do Daoo\Aula01Projeto02\classes\Atleta===
Nome: Luizito
Idade: 36
Pessoa: 80
Altura: 1.8
IMC: 24.691
```

log:

```
=== Dados de Daoo\Aula01Projeto02\classes\Medico ===
Herança: Daoo\Aula01Projeto02\classes\Abstracts\Pessoa
Atributos:
```

```
Nome: Pualo Paixão
Especialidade: Fisioterapeuta
343
```

```
destruido!
ruido!
```

```
13 },
14 "scripts": {
15     "start": "php -S localhost:8081 -t src",
16     "psy": "./vendor/psy/psysh/bin/psysh"
17 }
```

Herança Múltipla com Traits

Revisão de conceitos básicos de POO aplicados ao PHP.



- Nativamente o PHP não implementa herança múltipla diretamente de outras classes.
- O PHP suporta heranças múltiplas apenas utilizando o artifício das **Traits**.
- **Traits** são como classes utilizadas apenas para serem compartilhadas.
- Ou seja, compartilhamos seus recursos como propriedades e métodos.
- Uma classe pode usar diversas **Traits**.
- Por exemplo, a classe IMC da atividade anterior também poderia ser uma **Trait**, usada pela classe **Atleta**.

Herança Múltipla com Traits

Revisão de conceitos básicos de POO aplicados ao PHP.



- Exemplo da criação de uma **Trait**: transformando a **IMC** em Trait.

```
COMPOSER_PROJECT
├── src
│   ├── app
│   ├── classes
│   ├── interfaces
│   │   └── IMC.php
│   └── traits
│       └── IMC.php
├── .env
├── exemplo_01_spl_autoload.p...
├── exemplo_02_autoload_psr4....
├── exemplo_03_utils_var_dum...
├── exemplo_04_abstract.php
├── exemplo_05_interface.php
├── exemplo_06_exception.php
└── exemplo_07_pdo_database

src > traits > IMC.php > PHP Intelephense > {} IMC
1  <?php
2
3  namespace Daoo\Aula01Projeto02\traits;
4
5  trait IMC
6  {
7      protected float | null $imc;
8
9      public function calcIMC(): void
10     {
11         if(isset($this->peso)&&isset($this->altura)) {
12             $this->imc = $this->peso/$this->altura**2;
13         } else {
14             echo "Erro, defina peso e altura primeiro!";
15         }
16     }
```

Herança Múltipla com Traits

Revisão de conceitos básicos de POO aplicados ao PHP.



- Exemplo de uso de uma **Trait**: classe **Jogador** herda da classe **Pessoa** e usa a Trait **IMC**. Os métodos **calclmc** e **showlmc** agora estão na **Trait IMC**.

```
COMPOSER_PROJECT  src > classes > Jogador.php > PHP Intelephense > Jogador

src
├── app
├── classes
│   ├── Abstracts
│   ├── database
│   ├── logs
│   ├── Arbitro.php
│   ├── Atleta.php
│   ├── Atletas.php
│   ├── Jogador.php
│   └── Medico.php
└── ...

1  <?php
2  namespace Daoo\Aula01Projeto02\classes;
3
4  use Daoo\Aula01Projeto02\classes\Abstracts\Pessoa;
5  use Daoo\Aula01Projeto02\traits\IMC;
6
7  class Jogador extends Pessoa{
8
9      use IMC;
10
11     public $altura, $peso;
12
```


Herança Múltipla com Traits

Revisão de conceitos básicos de POO aplicados ao PHP.



- No construtor da classe **Jogador** eu tenho acesso as propriedades e métodos da trait **IMC**, ***\$this->imc*** e ***\$this->calcImc()***

src > classes >  Jogador.php > PHP Intelephense >  Jogador

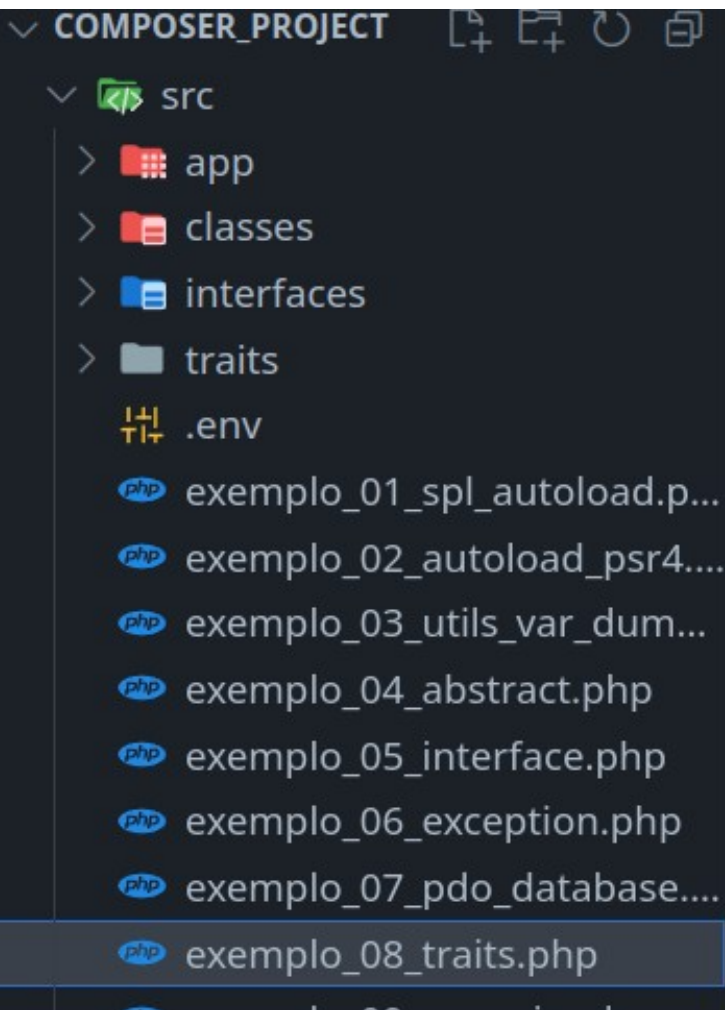
```
7  class Jogador extends Pessoa{
12
13      public function __construct($nome, $idade, $altura,$peso)
14      {
15          $this->nome = $nome;
16          $this->idade = $idade;
17          $this->altura = $altura;
18          $this->peso = $peso;
19          $this->calcImc();
20      }
```

Herança Múltipla com Traits

Revisão de conceitos básicos de POO aplicados ao PHP.



- Executando um exemplo:



```
src > exemplo_08_traits.php > ...
1  <?php
2
3  echo "<pre>";
4
5  use Daoo\Aula01Projeto02\classes\Jogador;
6  use Daoo\Aula01Projeto02\classes\Medico;
7  use Daoo\Aula01Projeto02\classes\logs\Relatorio;
8
9  $atl1 = new Jogador("Pedro Geromel",36,1.83,75);
10 $med1 = new Medico("Pualo Paixão",122343,"Fisioterapeuta");
11
12 $relatorio = new Relatorio;
13 $relatorio->add($med1);
14 $relatorio->add($atl1);
15 $relatorio->imprime();
```

Herança Múltipla com Traits

Revisão de conceitos básicos de POO aplicados ao PHP.



- Exemplo de uso de **Traits** em um *framework* como o **Laravel**:

```
1 <?php
2
3 namespace App\Models;
4
5 use Illuminate\Contracts\Auth\MustVerifyEmail;
6 use Illuminate\Database\Eloquent\Factories\HasFactory;
7 use Illuminate\Foundation\Auth\User as Authenticatable;
8 use Illuminate\Notifications\Notifiable;
9 use Laravel\Sanctum\HasApiTokens;
10
11 class User extends Authenticatable
12 {
13     use HasApiTokens, HasFactory, Notifiable;
14 }
```

Interface em PHP

Usando uma interface em PHP



- O que é uma Interface?

É uma estrutura que determina quais métodos uma classe deve implementar. No entanto, uma interface não define como estes métodos serão implementados, apenas a assinatura e o retorno de cada método.

- É possível implementar mais de uma interface.

- **Documentação!**

```
1  <?php
2
3  // Declara a interface 'iTemplate'
4  interface iTemplate
5  {
6      public function setVariable($name, $var);
7      public function getHtml($template);
8  }
9
10 // Implementa a interface
11 class Template implements iTemplate
12 {
```


Interface vs Classe Abstrata

Primeiros conceitos



- E qual a diferença entre **Interface** e classe **Abstrata** ?

As duas estruturas são utilizadas de forma **completamente diferente** e **não podem** ser uma **alternativa** de uma para a outra.

A confusão surge por que classes *Abstratas* também possuem métodos “vazios”, ou seja, ***métodos abstratos***, sem conteúdo, apenas **assinatura**!

No entanto, são apenas métodos **específicos** que os **Filhos** da classe **Abstrata** devem possuir na sua própria **implementação**.

A classe *Abstrata* possui **propriedades** e **métodos** comuns aos seus herdeiros (**filhos**) e **implementados** nela mesma.

Enquanto que uma **Interface** não possui **NENHUMA IMPLEMENTAÇÃO!**



- Por que uma Interface?

Neste exemplo, qualquer Classe que implementar a **interface IMC**, garantirá as demais classes do sistema a disponibilidade do uso dos métodos **calcImc** e **showImc**.

```
src > interfaces >  IMC.php > ...  
1  <?php  
2  
3  namespace Daoo\Aula01Projeto02\interfaces;  
4  
5  interface IMC  
6  {  
7      public function calcImc(): void;  
8      public function showImc(): void;  
9  }  
10
```

Interface em PHP

Usando uma interface em PHP



- Exemplos:

src >  exemplo_05_interface.php > PHP Intelephense >  esperaObjetoQueCalculaIMC

```
4
5  use Daoo\Aula01Projeto02\classes\Arbitro;
6  use Daoo\Aula01Projeto02\classes\Atleta;
7  use Daoo\Aula01Projeto02\interfaces\IMC;
8
9  function esperaObjetoQueCalculaIMC(IMC $pessoa){
10      $pessoa->showImc();
11  };
12
13  $atl1 = new Atleta("Walter Kannemann",33,1.84,83);
14  $arbi = new Arbitro("Anderson Daronco",43,'Juiz',1.88,90);
15
16  esperaObjetoQueCalculaIMC($atl1);
17  esperaObjetoQueCalculaIMC($arbi);
```

Interface em PHP

Usando uma interface em PHP



- **Exemplos:** Atleta e Arbitro implementa os métodos da interface IMC de formas diferentes.

src > classes > Atleta.php > PHP Intelephense > Atleta > __construct

```
9  class Atleta extends Pessoa implements IMC{
39      public function showImc():void
40      {
41          if(is_numeric($this->imc))
42              echo "\n0 IMC do $this->nome é: " . number_format($this->imc, 2) . "\n";
43      }
44
```

src > classes > Arbitro.php > PHP Intelephense > Arbitro

```
9  class Arbitro extends Pessoa implements IMC{
39      public function showImc():void
40      {
41          if(is_numeric($this->imc))
42              echo "\n0 IMC do $this->cargo $this->nome é: " . number_format($this->imc, 1) . "\n";
43      }
44
```

Bloco Try/Catch e Exceções

Tratamento de exceções.



- Blocos ***Try/Catch*** no PHP são semelhantes ao padrão de linguagens orientadas a objetos.
- Portanto as palavras reservadas ***try***, ***catch*** e ***finally*** possuem as mesmas funcionalidades de tratamento de erros de outras linguagens POO.
- Toda vez que é necessário tratar um erro, esse erro deverá gerar um objeto de Exceção (***\Exception***), sendo lançado pela palavra reservada ***throw***.
- O bloco ***finally*** será sempre executado;
- Se uma exceção ocorrer e não for capturada, esta será repassada a pilha de chamada de funções até encontrar um bloco de captura de exceção compatível, ou seja a exceção é repassada.
- ***Documentação Exceções;***

Bloco Try/Catch e Exceções

Tratamento de exceções.



- Exemplo sem tratamento de exceção:

```
1 public function calc() : float {
2     if ($this->peso && $this->altura) {
3         $this->imc = $this->peso/$this->altura**2;
4         return $this->imc;
5     }else{
6         echo "Erro, informe o peso e a altura primeiro!";
7         return 0;
8     }
9 }
```


Bloco Try/Catch e Exceções

Tratamento de exceções.



- Exemplo com tratamento de exceção:

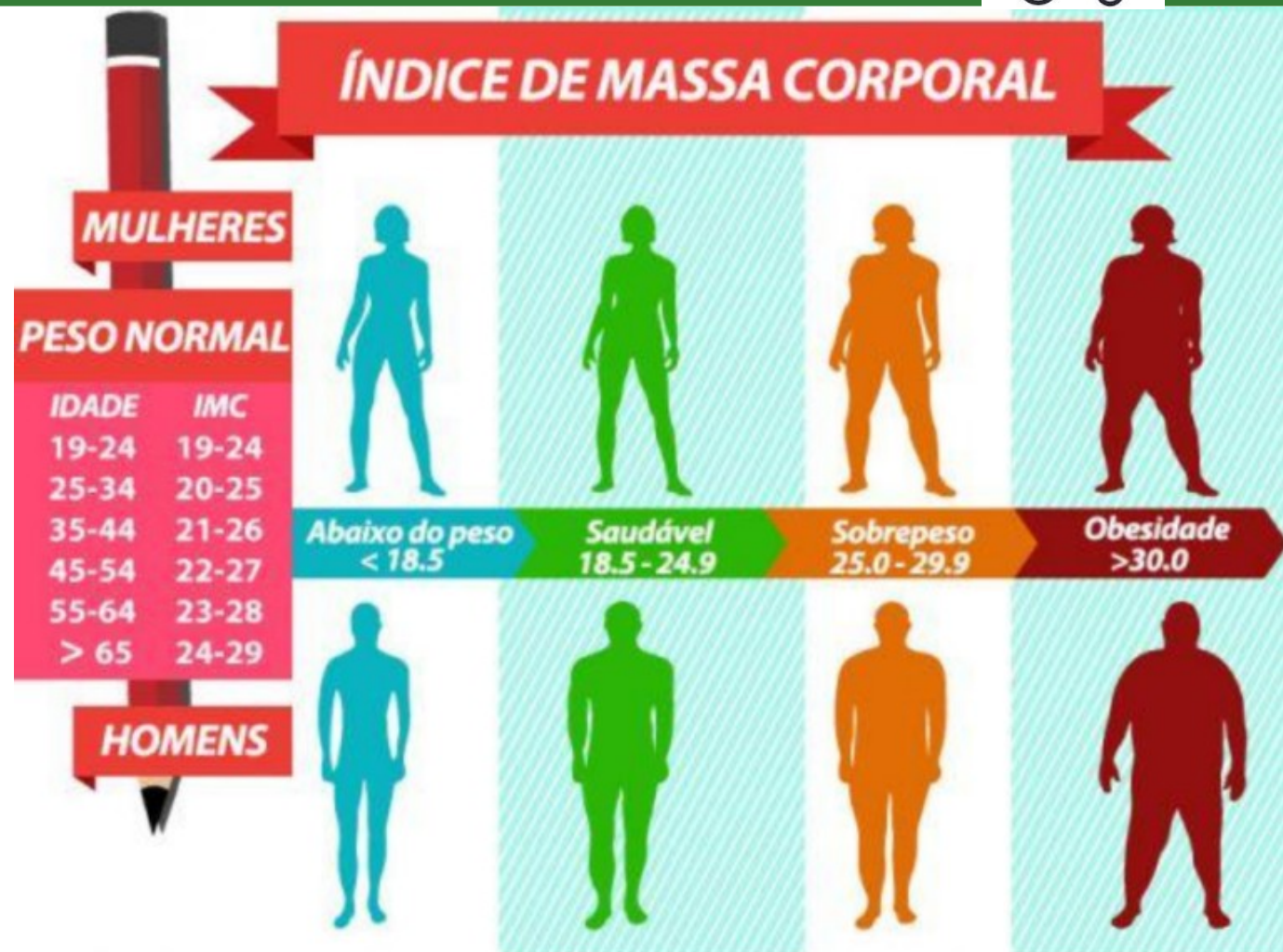
```
src > classes >  Atleta.php > PHP Intelephense >  Atleta
 9  class Atleta extends Pessoa implements IMC{
24      public function calcImc():void
26          try {
27              if(isset($this->peso)&&isset($this->altura)) {
28                  $this->imc = $this->peso/$this->altura**2;
29              } else{
30                  throw new Exception("Erro, defina peso e altura primeiro!");
31              }
32          } catch (Exception $error) {
33              echo $error->getMessage();
34              foreach($error->getTrace() as $trace) print_r($trace);
35              throw $error;
36          }
37      }
```

Atividade de Revisão

Revisão de conceitos básicos de POO aplicados ao PHP.



- **ATIVIDADE 2.1:** Crie a classificação recomendada ao lado através de uma **Trait** chamada **IMC**. A classe Atleta deverá usar esta trait e herdar os seguintes métodos:
- **calcIMC:** calcula o IMC do atleta, alterando o atributo **\$this->imc**.
- **classifica:** Calcula e retorna a classificação do IMC como **string**.
- **isNormal:** Retorna **true** se o **imc** é normal considerando a idade.



FONTE: <https://fs21.com.br/studiofs21/2020/08/15/228/>

Atividade de Revisão

Revisão de conceitos básicos de POO aplicados ao PHP.



- **ATIVIDADE 2.2:** Implemente a **INTERFACE iFuncionario**, com os seguintes métodos:
 - **mostrarSalario:** não recebe parâmetros e retorna a **string** “Salário: R\$ 00.000,00”
 - **mostrarTempoContrato:** não recebe parâmetros e retorna o tempo total de contrato como **string**, exemplo “Contrato de x anos.”.
 - A interface deverá ser implementada pelas classes Atleta ou Médico, que continuarão sendo do tipo **Pessoa**.
 - Durante a execução do método **log** da classe **Relatório**, as informações de contrato serão apresentadas apenas para aquelas classes que implementem a interface **iFuncionario**. Não modifique a assinatura do método **add**, ele deverá seguir recebendo um objeto do tipo **Pessoa**.
 - Para saber se uma classe implementa ou não uma interface use o operador de tipos **instanceof** do php, que retorna um booleano.

Atividade de Revisão

Revisão de conceitos básicos de POO aplicados ao PHP.

- Exemplo:

```
1 <?php
2 require "autoload.php";
3
4 use classes\Atleta;
5 use classes\Medico;
6 use logs\Relatorio;
7
8 $medico = new Medico("Paulo Paixão",12345,"Fisiologista",65);
9 $jogador = new Atleta("Pedro Geromel", 1.9, 84);
10
11 $relatorio = new Relatorio;
12 $relatorio->add($medico);
13 $relatorio->add($jogador);
14 $relatorio->imprime();
```

- \$ php exemplo-08-abstract-class.php

RELATORIO

log:
===Dados de classes\Medico ===
Nome: Paulo Paixão
Idade: 65
Especialidade: Fisiologista
CRM: 12345

log:

===Dados do Atleta===
Nome: Pedro Geromel
Peso: 84
Altura: 1.9
IMC:
- Valor: 23.27
- Classificacao: Saudável
=====

Contrato:

Salário: R\$ 45000,00
Contrato de 4 anos.
#####

Pedro Geromel foi destruído!

Paulo Paixão foi destruído!



- LINKS SOBRE PDO:
 - <https://www.php.net/manual/en/book.pdo.php>
 - <https://www.php.net/manual/en/pdo.drivers.php>
 - <https://www.php.net/manual/en/ref.pdo-mysql.connection.php>
 - <https://www.php.net/manual/en/ref.pdo-pgsql.connection.php>
 - https://www.php.net/manual/pt_BR/language.oop5.interfaces.php
 - https://www.php.net/manual/pt_BR/language.oop5.decon.php
- DALL'OGGIO, Pablo. **Php: programando com orientação a objetos**. 2. ed. São Paulo: Novatec, 2009